

FATEC POMPEIA

Introdução à Computação em Nuvem — ISW055

P1

Relatório Bimestral de Atividades

Avaliação Individual — 2026.2

Lucas Hiratuca

Prof. Allan Lincoln Rodrigues Siriani

Pompeia, 07/10/2026

Sumário

1 Introdução

2 Metodologia

3 Quadro de Entregas

4 Atividades Realizadas

4.1 Atividade 1 — Agenda Telefônica em Flask

4.2 Atividade 2 — Catálogo de Filmes — Tom Hanks

4.3 Atividade 3 — Desacoplando o Login — Microsserviço de Autenticação

4.4 Atividade 4 — Controle de Acesso por Papel — RBAC

4.5 Atividade 5 — Logs e Auditoria

4.6 Atividade 6 — Upload e Perfil de Usuário (Garage S3)

5 Considerações Finais

6 Declaração de Autoria

7 Guia de Prints — O Que o Aluno Deve Capturar

1 Introdução

A disciplina **ISW055 — Introdução à Computação em Nuvem**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na FATEC Pompeia no segundo semestre de 2026, tem como proposta central conduzir o aluno por uma jornada incremental de desenvolvimento de software orientado à nuvem: partindo de uma aplicação monolítica simples, o projeto evolui, atividade após atividade, até uma arquitetura real de microsserviços com autenticação, controle de acesso, auditoria e armazenamento de objetos em nuvem.

O fio condutor do bimestre foi o **Catálogo de Filmes de Tom Hanks** — uma aplicação web construída em Node.js/Express/EJS que consome a API pública do TMDb para exibir a filmografia completa do ator. Sobre essa base, cada atividade acrescentou uma camada de funcionalidade e segurança: cadastro e login com JWT, favoritos por usuário, comentários moderados, controle de acesso baseado em papel (RBAC), auditoria com Redis Streams e, por fim, perfil de usuário com upload de foto em object storage distribuído (Garage S3).

Este relatório documenta todas as seis atividades entregues, com datas, commits verificáveis no GitHub, descrição das decisões técnicas e indicação das evidências (prints) que comprovam o funcionamento de cada entrega. O repositório principal do projeto é **LucasHiratuca/Entrega-da-Atividade** (<https://github.com/LucasHiratuca/Entrega-da-Atividade>).

2 Metodologia

Todas as atividades foram desenvolvidas em repositórios públicos no GitHub, com controle de versão via Git. A data e a hora de cada entrega foram extraídas do **histórico de commits** (git log) e conferidas na página do commit no GitHub, onde o timestamp é exibido publicamente e imutável.

O ambiente de desenvolvimento utilizado foi **Docker Desktop no Windows 10/11**, com orquestração via docker compose. O deploy em produção é realizado via **Portainer** no servidor do professor (lucas-hiratuca-isw055.lapps.studio), exposto com Cloudflare Tunnel. As imagens Docker são publicadas no **Docker Hub** sob o namespace lucashiratuca/.

Os prints foram coletados a partir das interfaces web em execução local (localhost:8217) ou em produção, mostrando: o README do repositório no GitHub com a menção ao professor, o sistema em funcionamento e, quando aplicável, o resultado de testes de segurança.

3 Quadro de Entregas

Nº	Atividade	Data Planejada	Data Realizada	Situação
1	Agenda telefônica em Flask	07/08/2026	07/08/2026 20:03	entregue
2	Catálogo de filmes — Tom Hanks	20/08/2026	20/08/2026 20:54	entregue
3	Microsserviço de autenticação	28/08/2026	27/08/2026 13:00	entregue
4	Controle de acesso por papel — RBAC	04/09/2026	01/09/2026 00:48	entregue
5	Logs e auditoria com Redis	25/09/2026	08/09/2026 12:57	entregue
6	Upload e perfil — Garage S3	02/10/2026	26/09/2026 14:43	entregue

Todas as datas foram extraídas do histórico de commits do Git e são verificáveis publicamente nas páginas de commit do repositório no GitHub.

4 Atividades Realizadas

4.1 Atividade 1 — Agenda Telefônica em Flask

Campo	Valor
Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja2 com persistência em banco de dados MySQL
Data planejada	07/08/2026
Data realizada	07/08/2026 20:03 (Commit 28230dc)
Situação	entregue
Evidência	Repositório público no GitHub com histórico de commits e README com menção ao professor.
Link	https://github.com/LucasHiratuca/aula_01_cloud

A Atividade 1 foi realizada integralmente durante a aula de nivelamento do dia 07/08/2026. Desenvolveu-se uma aplicação web em **Python/Flask** com templates **Jinja2** para cadastro e consulta de contatos telefônicos, com persistência em **MariaDB hospedado na nuvem** (servidor do professor em 35.226.64.52). O sistema foi versionado no repositório LucasHiratuca/aula_01_cloud e inclui: formulário de cadastro, listagem de contatos e conexão via PyMySQL.

Decisão técnica relevante: optou-se por utilizar variáveis de ambiente para as credenciais do banco de dados (DSN), evitando expô-las no código-fonte. O commit de fix fa5f58d (14/09/2026) removeu credenciais que tinham sido expostas acidentalmente e adicionou .gitignore.

[PRINT 1A — Atividade 1 — Evidência: README do repo LucasHiratuca/aula_01_cloud no GitHub com commit visível e menção ao professor]
Veja as instruções ao final deste documento.

[PRINT 1B — Atividade 1 — Resultado: tela da agenda telefônica em Flask com um contato cadastrado e listado]
Veja as instruções ao final deste documento.

Dificuldades: A configuração da conexão com o banco de dados remoto exigiu ajustes na string de conexão e no tratamento de encoding. Resolvida consultando a documentação do PyMySQL e ajustando o charset para utf8mb4.

4.2 Atividade 2 — Catálogo de Filmes — Tom Hanks

Campo	Valor
Descrição	Consumo da API TMDb para exibir a filmografia de Tom Hanks, persistência em MariaDB e segreg
Data planejada	20/08/2026
Data realizada	20/08/2026 20:54 (Commit 9a57391)
Situação	entregue
Evidência	GitHub — commit 9a57391 + README com menção ao professor + print do catálogo em funcion
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/9a57391

Nesta atividade foi criada a aplicação principal do projeto: um catálogo web em **Node.js/Express** com templates **EJS**, consumindo a **API REST do TMDb** para listar todos os filmes de Tom Hanks com poster, título, ano e nota. O sistema inclui cadastro e login de usuários com senhas em **bcrypt** e sessões com **express-session**, além de um sistema de **favoritos** persistido em MariaDB por usuário autenticado.

A aplicação foi containerizada desde o início com **Docker** e **docker-compose.yml**, exposta na porta 8217. O banco de dados MariaDB na nuvem foi acessado diretamente pelo **catalogo-service**, com inicialização automática das tabelas no startup.

[PRINT 2A — Atividade 2 — Evidência: página do commit 9a57391 no GitHub (URL + timestamp visíveis)]

Veja as instruções ao final deste documento.

[PRINT 2B — Atividade 2 — Resultado: tela do catálogo Tom Hanks com grid de filmes e posters carregados da API TMDb]

Veja as instruções ao final deste documento.

Dificuldades: A paginação da API TMDb retorna 20 filmes por página; foi necessário concatenar múltiplas requisições para obter a filmografia completa. Resolvido com um loop de fetch que acumula todas as páginas disponíveis.

4.3 Atividade 3 — Desacoplando o Login — Microserviço de Autenticação

Campo	Valor
Descrição	Separação do serviço de autenticação (login, cadastro, esqueci-minha-senha) num microserviço i
Data planejada	28/08/2026
Data realizada	27/08/2026 13:00 (Commit 320028a)
Situação	entregue
Evidência	GitHub — commit 320028a + docker-compose.yml com auth-service sem portas públicas + print do
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/320028a

O sistema foi refatorado para o padrão **BFF (Backend For Frontend)**: o catalogo-service passou a ser o único serviço com porta exposta publicamente, enquanto o novo auth-service — executando na porta 4000 — opera **exclusivamente na rede interna Docker** (app_network), inacessível pela internet.

O auth-service emite **tokens JWT** assinados com segredo compartilhado, gerencia as tabelas usuarios e reset_tokens no MariaDB, e envia e-mails reais de recuperação de senha via **Mailtrap** (SMTP). O catalogo-service valida o JWT a cada requisição protegida, sem tocar no banco de usuários diretamente.

[PRINT 3A — Atividade 3 — Evidência: commit 320028a no GitHub com docker-compose.yml mostrando auth-service sem ports públicas]
Veja as instruções ao final deste documento.

[PRINT 3B — Atividade 3 — Resultado: tela de login do catálogo com autenticação funcionando (acesso ao catálogo após login)]
Veja as instruções ao final deste documento.

Dificuldades: A comunicação entre serviços via rede interna Docker exigiu entender a resolução de nomes por hostname de container. O token JWT precisou incluir o campo userId para identificar o usuário nas rotas do catálogo.

4.4 Atividade 4 — Controle de Acesso por Papel — RBAC

Campo	Valor
Descrição	O campo role no JWT decide permissões reais no backend: HTTP 403 para usuário comum tentando acessar painel de administração.
Data planejada	04/09/2026
Data realizada	01/09/2026 00:48 (Commits 6636f4d e a8e31ef)
Situação	entregue
Evidência	GitHub — commits 6636f4d e a8e31ef + print do HTTP 403 e do painel admin funcionando.
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/6636f4d

Foi implementado o **Padrão B de RBAC**: as claims de papel (role) são embutidas diretamente no JWT pelo auth-service, permitindo que o catalogo-service tome decisões de autorização instantaneamente, sem consultas adicionais ao banco de dados.

O middleware requireAdmin foi criado para proteger as rotas /admin/*. Usuários com role: 'usuario' recebem **HTTP 403 Forbidden** ao tentar acessar: painel de usuários (/admin/usuarios), moderação de comentários ou visualização de logs. Administradores têm acesso exclusivo a alterar o papel de outros usuários e excluir comentários alheios. O controle é realizado **100% no backend**, independentemente da interface.

[PRINT 4A — Atividade 4 — Evidência: commit 6636f4d ou a8e31ef no GitHub com timestamp visível]
Veja as instruções ao final deste documento.

[PRINT 4B — Atividade 4 — Resultado: tela do painel admin (/admin/usuarios) e/ou tela de erro 403 ao tentar acessar como usuário comum]
Veja as instruções ao final deste documento.

Dificuldades: Garantir que o campo userId no JWT fosse sempre um inteiro (e não string) para comparação correta no backend. Resolvido com parseInt() explícito nas comparações de autorização.

4.5 Atividade 5 — Logs e Auditoria com Redis

Campo	Valor
Descrição	Novo log-service com Redis 7 registrando eventos de login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	08/09/2026 12:57 (Commits e6c6682 e 8bc9faa)
Situação	entregue
Evidência	GitHub — commits e6c6682 e 8bc9faa + print da consulta de logs pelo admin em /admin/logs.
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/e6c6682

Foi criado um terceiro microsserviço dedicado, o log-service (porta 5000), conectado a uma instância do **Redis 7**. O serviço expõe uma API REST interna que recebe eventos de auditoria via HTTP POST e os persiste em **Redis Streams** — a estrutura de dados ideal para logs imutáveis e sequenciais.

O catalogo-service dispara logs assincronamente para o log-service em cada evento relevante: login com sucesso, logout, falha de autenticação, exclusão de comentário e edição de perfil. A rota /admin/logs lista os eventos em ordem cronológica inversa, protegida pelo middleware requireAdmin.

[PRINT 5A — Atividade 5 — Evidência: commit e6c6682 no GitHub com timestamp visível]
Veja as instruções ao final deste documento.

[PRINT 5B — Atividade 5 — Resultado: tela /admin/logs mostrando eventos de auditoria em tempo real (login, logout, etc.)]
Veja as instruções ao final deste documento.

Dificuldades: A persistência dos logs no Redis exigiu entendimento da API de Streams (XADD / XRANGE). Inicialmente os logs somem ao recriar o container por falta de volume; resolvido declarando um volume nomeado para o Redis no compose.

4.6 Atividade 6 — Upload e Perfil de Usuário (Garage S3)

Campo	Valor
Descrição	Página de perfil com avatar no Garage S3; o banco relacional guarda só a referência (foto_key). A
Data planejada	02/10/2026
Data realizada	26/09/2026 14:43 (Commits 4219f05, 27856a8, 8ca3fb6)
Situação	entregue
Evidência	GitHub — commits 4219f05, 27856a8, 8ca3fb6 + print do perfil com foto carregada + relatório SEC
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/27856a8

Foi implementada a página de perfil de usuário com upload de foto armazenada no **Garage S3** (dxflrs/garage:v1.0.1), um object storage distribuído escrito em Rust, 100% compatível com a API S3 da AWS (Signature V4). O banco de dados MariaDB passou a armazenar apenas a **chave do objeto** (foto_key) na nova tabela perfis, jamais o binário da imagem.

A exibição da foto é feita via **streaming seguro pelo backend**: a rota /perfil/:userId/foto do catalogo-service recupera o objeto do Garage S3 internamente e retransmite os bytes para o navegador com os cabeçalhos corretos (Content-Type e Cache-Control: public, max-age=3600). O bucket permanece 100% privado na rede interna do Docker.

O upload é validado com **whitelist de tipos MIME** (jpeg, png, gif, webp), limite de **5MB** via Multer e nomes de objeto gerados com randomUUID() para prevenir path traversal. O controle de acesso garante que **somente o dono do perfil** pode editar — tentativas de editar perfis alheios retornam **HTTP 403** no backend.

Ainda nesta atividade foi realizada uma auditoria completa de segurança seguindo a metodologia **OWASP Top 10:2021**, com suíte de testes automatizados (owasp-security-suite.js) e relatório detalhado em SECURITY_AUDIT.md. Duas vulnerabilidades foram identificadas e corrigidas: **CWE-640 Password Reset Poisoning** e **CWE-614 Session Cookie sem flags**.

[PRINT 6A — Atividade 6 — Evidência: commit 27856a8 no GitHub com timestamp visível e lista de arquivos alterados]

Veja as instruções ao final deste documento.

[PRINT 6B — Atividade 6 — Resultado: tela /perfil/ com foto de avatar carregada e bio exibida]

Veja as instruções ao final deste documento.

Dificuldades: A imagem oficial do MinIO foi removida do Docker Hub em outubro de 2025, exigindo migração para o Garage S3. A exibição da foto via URL pré-assinada falhou porque a URL continha o hostname interno Docker (minio:9000), inacessível pelo navegador. A solução foi implementar a rota de streaming no backend, que resolve o problema definitivamente em qualquer ambiente (local, Portainer, Cloudflare).

5 Considerações Finais

O primeiro bimestre de ISW055 foi uma experiência intensa e muito produtiva. Partir de uma aplicação monolítica Flask e chegar a uma arquitetura de microsserviços com autenticação JWT, RBAC, auditoria em tempo real e object storage distribuído foi um salto técnico considerável. Cada atividade foi entregue antes ou na data planejada, o que demonstra que o ritmo proposto pelo professor foi bem absorvido.

A parte mais desafiadora foi a **Atividade 6**: a migração inesperada do MinIO para o Garage S3 (por remoção da imagem oficial do Docker Hub) e a depuração do problema das URLs internas do Docker. A solução de streaming pelo backend acabou sendo mais elegante do que a URL pré-assinada original, pois mantém o bucket privado e funciona em qualquer infraestrutura.

O que levo para o próximo bimestre: a necessidade de testar a aplicação em cenários adversariais desde o início (security-first design). A auditoria OWASP revelou vulnerabilidades sutis (como o Password Reset Poisoning) que só foram descobertas por uma revisão sistemática do código. No P2, pretendo incorporar testes automatizados de segurança no próprio pipeline de desenvolvimento.

6 Declaração de Autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Lucas Hiratuca

Pompeia, 07/10/2026

7 Guia de Prints — O Que Você Deve Capturar

Insira os prints diretamente neste PDF (ou numa versão Word/Google Docs) nos espaços indicados pelos blocos cinzas ao longo do documento. Cada print deve ser feito em alta resolução (recomendado: zoom do browser em 100%, resolução $\geq 1280 \times 720$).

Print 1A — Atividade 1 — Evidência

1. Abra o browser.
2. Acesse: https://github.com/LucasHiratuca/aula_01_cloud
3. Role até a seção de commits. Clique em 'commits'.
4. Capture a tela mostrando o commit '28230dc' com data '07/08/2026' e o README com o nome do professor visível.
5. A URL da página deve estar visível na barra do browser.

Print 1B — Atividade 1 — Sistema Funcionando

1. Suba localmente o projeto da Atividade 1 (aula_01_cloud).
2. Acesse <http://localhost:5000> (ou a porta configurada).
3. Cadastre um contato e faça uma busca.
4. Capture a tela mostrando o formulário preenchido e a lista de contatos.

Print 2A — Atividade 2 — Evidência

1. Acesse: <https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/9a57391>
2. Capture a página inteira mostrando: hash do commit, data '20/08/2026 20:54', lista de arquivos alterados e a URL na barra do browser.

Print 2B — Atividade 2 — Sistema Funcionando

1. Suba a stack: `docker compose up -d --build` (no diretório `catalogo-tom-hanks`)
2. Acesse: <http://localhost:8217/filmes> (faça login antes)
3. Capture a tela com o grid de filmes do Tom Hanks, posters carregados e a navbar visível.

Print 3A — Atividade 3 — Evidência

1. Acesse: <https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/320028a>
2. Capture mostrando: data '27/08/2026 13:00', e o diff do `docker-compose.yml` com o `auth-service` SEM ports publicas.

Print 3B — Atividade 3 — Sistema Funcionando

1. Com a stack subida, acesse: <http://localhost:8217/login>
2. Faça login com um usuário cadastrado.
3. Capture a tela após o login mostrando que está autenticado (nome do usuário na navbar).

Print 4A — Atividade 4 — Evidência

1. Acesse: <https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/6636f4d>
2. Capture mostrando: data '01/09/2026 00:48' e os arquivos modificados.

Print 4B — Atividade 4 — RBAC em Ação

Opção A (403): Faça login como usuário COMUM. Tente acessar <http://localhost:8217/admin/usuarios>. Capture a tela de erro HTTP 403.

Opção B (Admin): Faça login como ADMIN. Acesse [/admin/usuarios](#). Capture a lista de usuários com a opção de alterar roles.

Print 5A — Atividade 5 — Evidência

1. Acesse: <https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/e6c6682>
2. Capture mostrando: data '08/09/2026 12:57' e lista de arquivos (log-service/).

Print 5B — Atividade 5 — Logs em Ação

1. Faça login como ADMIN.
2. Acesse: <http://localhost:8217/admin/logs>
3. Capture a tela mostrando os eventos de auditoria listados (login, logout, edição de perfil, etc.).

Print 6A — Atividade 6 — Evidência

1. Acesse: <https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/27856a8>
2. Capture mostrando: data '26/09/2026 15:11', lista de arquivos (garage.toml, s3.js, perfil.js).

Print 6B — Atividade 6 — Perfil com Foto

1. Com a stack subida e o Garage S3 inicializado (.\ init-garage.ps1), faça login.
2. Acesse: <http://localhost:8217/perfil/editar>
3. Faça upload de uma foto (JPEG/PNG) e salve.
4. Acesse: <http://localhost:8217/perfil/>
5. Capture a tela mostrando o avatar carregado, a bio e a grade de favoritos.

■ **Após inserir todos os prints, salve o arquivo como PDF e envie conforme instrução do professor.**